

Document Number: P1896R0
Date: 2019-10-02
Audience: SG16
Reply-to: Tom Honermann <tom@honermann.net>

SG16: Unicode meeting summaries 2019/06/12 - 2019/09/25

Summaries of SG16 meetings are maintained at <https://github.com/sg16-unicode/sg16-meetings>. This paper contains a snapshot of select meeting summaries from that repository.

- [June 12th, 2019](#)
- [June 26th, 2019](#)
- [July 31st, 2019](#)
- [August 21st, 2019](#)
- [September 14th, 2019](#)
- [September 25th, 2019](#)

June 12th, 2019

Draft agenda:

- Discuss and provide feedback for any draft papers targeting the 6/17 pre-Cologne mailing.

Attendees:

- Nathan Myers
- JeanHeyd Meneide
- Mark Zeren
- Steve Downey
- Tom Honermann
- Zach Laine

Meeting summary:

- Planning for Cologne:
 - Tom communicated that SG16 has requested a half day session in Cologne.
 - Tom communicated that SG16 will host an evening session. Potential topics (subject to author's desire) include:
 - UTF-8 and current ecosystems.
 - JeanHeyd's work on transcoding interfaces.
 - Corentin's work on character properties.
 - Hana's work on Unicode support in CTRE.
 - JeanHeyd confirmed that his transcoding interfaces paper will appear in the pre-meeting mailing.
- Discussion of the file name constraints added to the draft D1238R1 posted to the SG16 mailing list:
 - <http://www.open-std.org/pipermail/unicode/2019-June/000386.html>
 - Steve expressed approval for the new section.

- Zach agreed noting uncertainty that anyone cares about the details of normalization-insensitivity.
- Tom concurred and indicated he was unsure how important that is.
- Zach stated that it is important since extremely subtle bugs can happen from changing normalization.
- Tom acknowledged the possibility and noted reported problems for Apple's migration from HFS+ to APFS.
- Zach observed that there is no good way to tell what filesystem you are working on and what its idiosyncracies are.
- Nathan asserted that programmers have to deal with presentation of file names and allow user selection.
- Steve noted that different file names can present the same (due to Unicode confusables or normalization differences).
- Zach recalled an email from Marshall Clow some time ago regarding file systems using completely different normalization schemes. Different filesystems do things differently.
- Nathan stated that uploading a file to a web site also has presentation issues.
- Mark stated that jumping from one filesystem to another is inherently lossy, but treated as a transfer issue. The only way to store a file accurately in text is to write it in something like base64. Writing a file name to a text file may break the encoding of the file.
- Zach claimed that we can't fix these issues except by declaring "things must work" and letting implementors figure it out, which they probably can't do.
- Steve noted that we keep getting asked about handling of file names and this is intended to document constraints.
- Mark recalled an example; from the stack trace proposal, we specified file names be handled as a sequence of bytes.
- Tom mentioned he was thinking about sending an email to the Unicode Consortium's mailing list asking about current thinking regarding file names in text files.
- Mark argued that we should just try and stay out of this space.
- Tom asserted it is a big question for `std::text``. How do we allow file names in `std::text`, particularly if we require well-formed content?
- Mark suggested relying on an error policy.
- Zach claimed that we need to emphasize that, if a file name is retrieved from the file system, programmers must maintain it as is. Don't mutate it at all, don't compare it to text.
- Tom asked how one puts file names in text and have it be well-formed text?
- Zach replied simply, you don't.
- Mark provided an example of Apache using base64 encoding of names in URLs.
- Zach asserted that applications must provide a file selector interface.
- Tom asked how one would write `ls`?
- Zach responded that the file name be written in a presentation format that isn't necessarily suitable for referencing the file.
- Steve observed that this already happens all the time that file names appear in output, but can't be parsed out or referenced as is.
- Tom acknowledged and observed this is why GNU `find`` has a `-print0` option.
- Nathan suggested that we may need to publish a document on how to deal with file names.
- Steve mentioned that we have `std::filesystem` and it has facilities for getting names out of paths.
- Zach claimed that problems happen if, for example, you have a UCS-2 file name on Windows that is ill-formed UTF-16.
- Tom confirmed that recent Windows 10 releases still allow creation of file names that are not valid UTF-16.
- Zach asserted that we don't want interfaces that do transcoding or normalization to touch filenames.
- JeanHeyd suggested adding a new non-directive to the paper stating that we won't attempt to impose restrictions on file names.
- Tom agreed to do so.
- *[Editor's note: Tom did so in [P1238R1](#) for the Cologne pre-meeting mailing]*
- Discussion of planned transcoding papers:

- Zach stated he wasn't going to be able to produce a paper on transcoding for the Cologne pre-meeting mailing.
- Tom let Zach know that was ok, especially since JeanHeyd was who had volunteered to write that paper and is currently working on a draft.
- Zach noted a performance concern to address in the paper; generic transcoder interfaces don't perform well with smart iterators. For maximum performance, vector operations must be used as Bob Steagall demonstrated.
- Tom acknowledged that specializations for contiguous storage are needed.
- JeanHeyd said he came to the same conclusion and that the paper would discuss it. He also indicated intent to share the draft on the SG16 mailing list.
- *[Editor's note: JeanHeyd later shared that draft on the SG16 Slack channel. The draft can be found at https://thephd.github.io/vendor/future_cxx/papers/d1629.html and will be in the pre-meeting mailing as [P1629R0](#).]*
- Discussion of z/OS compiler updates:
 - Tom communicated recent news within the z/OS ecosystem. IBM recently released versions of Clang for z/OS with their latest updates for their xLC compiler. Additionally, a third party provider also maintains a z/OS C++14+ compiler based on LLVM. Tom stated the details would appear in a revision of P1238.
 - *[Editor's note: Tom did add those details to [P1238R1](#) for the Cologne pre-meeting mailing]*
- Discussion of Boost review for JeanHeyd's [out_ptr](#).
 - Zach communicated that Boost formal review for JeanHeyd's [out_ptr](#) library would begin on Monday June 17th and encouraged everyone to participate via the Boost mailing list.
- Discussion of C standard string transcoding functions:
 - JeanHeyd asked for feedback regarding a set of transcoding functions he is considering proposing to the C committee at their October meeting in Ithaca. The functions match the existing C `mbstowcs/wcstombs` and `mbsrtowcs/wcsrtombs` functions but transcode between UTF-8 (`char8_t`), UTF-16 (`char16_t`), and UTF-32 (`char32_t`). The full cartesian product for all of the encodings results in approximately 40 functions. He is wondering if the full set is needed or if a reduced set would suffice. These functions are not used often and aren't very performant.
 - Tom stated that `mbstowcs` should be able to perform ok.
 - JeanHeyd stated that dropping the restartable ones would reduce the number, but those are useful in some cases. Another approach is to just propose the `c8`, `c16`, and `c32` variants that convert between the execution encoding.
 - Tom agreed that just providing conversion between the execution encoding and UTF variants was probably sufficient.
- Discussion of updates to [P1072](#):
 - Mark provided an update regarding plans for [P1072](#). There are two options:
 - Propose a lambda based interface.
 - Propose an independent class coupled to `std::string`.
 - Mark clarified that neither proposal would appear in the pre-meeting mailing.
 - Zach expressed a desire for the functionality and for additional progress to be made.
- Discussion of planned C committee proposals:
 - Tom asked for any volunteers interested in writing and presenting papers to the C committee in October that propose functionality we've added or plan to add for C++. Such features include:
 - `char8_t` ([P0482](#))
 - Make `char16_t/char32_t` string literals be UTF-16/32 ([P1041](#))
 - Named character escapes: ([P1097](#))
 - JeanHeyd asked if we had ever followed up with the C committee regarding any known implementations that use an encoding other than UTF-16/UTF-32 for `char16_t/char32_t` literals or that don't define `__STDC_UTF_16__` and/or `__STDC_UTF_32__`.
 - Tom responded that Philipp Krause had confirmed that there are no known implementations.
 - *[Editor's note: though not mentioned in the meeting, there are implementations that use UTF-16 and UTF-32, but neglect to define the `__STDC_UTF_16__` and/or `__STDC_UTF_32__` macros.]*

June 26th, 2019

Draft agenda:

- Discuss papers from the Cologne pre-meeting mailing. At least:
 - P1629R0 - Standard Text Encoding
 - P0267R9 - A Proposal to Add 2D Graphics Rendering and Display to C++
 - just the new interfaces for text rendering.

Attendees:

- Elias Kosunen
- Hubert Tong
- JeanHeyd Meneide
- JF Bastien
- Mark Zeren
- Michael Spencer
- Peter Bindels
- Steve Downey
- Tom Honermann
- Zach Laine

Meeting summary:

- Tom started the meeting with some administrative details:
 - Our regular meeting cadence would have us meet July 10th and July 24th, but the Cologne meeting is the 15th through the 20th. Tentative plan is to skip the next two regular meetings, meet July 31st, and then back to our regular meetings during the 2nd and 4th weeks of the month in August.
 - Hubert asked when the post-meeting mailing deadline is.
 - Mark responded, August 5th.
 - Tom communicated that issue #8 (<https://github.com/sg16-unicode/sg16/issues/8>) has been closed as resolved by the adoption of [P1139R2](#) in Kona.
 - Tom also communicated that the revision of [P1423R2](#) in the Cologne pre-meeting mailing adds deleted operator<< overloads for wide streams for char8_t, char16_t, and char32_t following LWG feedback during their [May 21st paper review telecon](#). These changes will require LEWG review in Cologne.
- [P1629R0 - Standard Text Encoding](#):
 - JeanHeyd presented and provided a link to a draft revision (with only clerical errors fixed).
 - https://the-phd.github.io/vendor/future_cxx/papers/d1629.html
 - The proposal includes low level and high level interfaces.
 - Normalization support will come later.
 - Peter (via chat): There is a typo in section 3.3.2, "GB1032" should be "GB2312" or "GB18030".
 - Elias (via chat): In 3.2.3.2, on the last line of the first snippet, the basic_utf8 instead of basic_utf16 is probably a typo?
 - Zach expressed surprise at the lack of low level transcoding algorithms and lack of iterator based interfaces.
 - JeanHeyd replied that those algorithms are implemented within the encoding object and that the interface is range based rather than iterator based. Objects are used instead of free functions in order to maintain state.
 - Zach asked where code point conversion is happening; there isn't much state needed.

- JeanHeyd explained that roundtripping through the encoding handles code points internally. State is needed for non-Unicode encodings and for error handling.
- Zach stated that, in Boost.text, the error handler is a template parameter.
- Zach asked if this design precludes doing performance optimizations like Bob Steagall has demonstrated.
- JeanHeyd replied that such optimizations are excluded in the encoder interface, but are intended to be supported by specializing the high level interfaces; the specified free functions are customization points that can enable optimizations.
- Tom asked why the encode and decode functions on the encoding object preclude optimizations.
- JeanHeyd replied that they only process one code point at a time.
- Zach asked what the motivation is for the slower interfaces over faster ones.
- JeanHeyd replied that the encode and decode customization points are eager and convert as much as possible. The encoding object enables an iterative approach in which writing just the encoding object suffices to enable the high level interfaces to work correctly, but at a less-than-optimal speed.
- Steve said that it sounds like the code point at a time encoding object is the extension point for custom encoding. It is unlikely that anyone will bother with a high performance implementation for many legacy encodings as vectorizing support takes a lot of work.
- Zach expressed support for a convenience approach and a fast path, but also sees value in an iterator approach as well. Encoding details should be in either the algorithm (eager/fast) or in the iterator (lazy/slow). Having building blocks for constructing iterators isn't key.
- Zach expanded by contrasting with Python where the encode and decode functions always confused him because encoding and decoding are basically different names for the same algorithm with direction reversed. This design seems over generalized.
- Tom stated that the design is range based, so iterators can be wrapped in a range, does that not suffice for iterator use cases?
- Zach replied that standard algorithms don't take an output range, they take output iterators.
- Zach stated, when I'm doing a transcode, sometimes I want to loop and break, sometimes I just want to convert everything.
- Peter stated he was confused by Zach's comments.
- JeanHeyd attempted to paraphrase. What Zach is saying, is rather than specify building blocks, we should specify lazy transcoding iterators. The concern with that approach is that writing an iterator is a lot harder to do.
- Tom agreed noting that he discovered how hard they are to write when working on text_view. For example, decoding iterators need to eagerly consume code units.
- Mark noted that we don't need to make it easy for implementors to write iterators, but it is good to make things easy for other programmers.
- Zach stated that someone still needs to write the lazy iterator. There is an impedance mismatch between input and output. A general template based iterator doesn't work.
- Tom stated it did for text_view.
- JeanHeyd stated that the ideas came from text_view and libogonek. The encoding object avoids having to write iterators and ranges.
- Zach stated he would like to understand how that works.
- Tom explained how input text iterators and output text iterators can be used together; e.g., via `std::copy`.
- JeanHeyd expounded; Libogonek proved this out and Peter's S2 library did something similar.
- Peter (via chat): +1, doing exactly that in <http://github.com/dascandy/s2>. I have the rope concept that combines different code-point iterators as a single range so you can copy from that to a target (and the assignment operator for target encodings is optimized to first calculate size & then do the copy).
`s2::basic_string<s2::encoding::utf8> u8s = u16s.view();`
- Peter (via chat): 90% sure this is my hook for encoding conversion fast path - https://github.com/dascandy/s2/blob/master/include/s2/detail/rope_detail.h#L41
- Zach said he would like to see the code in libogonek to better understand it. It is well understood how encoders produce code units and decoders produce code points, but hard to see how transcoding can be done without missing optimization opportunities.

- JeanHeyd explained that the fast path customization points enable that optimization by skipping the separate decode and encode steps.
- Zach asked if iterator facade ever got standardized? It makes writing iterators easy.
- *[Editor's note: no they haven't. The iterator facade proposal is [P0186](#). It was discussed in Oulu in 2016. Meeting minutes are [here](#).]*
- Zach expressed skepticism regarding encoding builders; we just need to worry about common encodings.
- Tom stated that there are use cases for code point at a time enumeration.
- Zach agreed but stated that should be provided via lazy iterators; this design is taking generic programming too far.
- Zach expressed a desire to be able to write a transcoding iterator that avoids construction of the intermediate code point value during conversion.
- JeanHeyd noted that there are three extension points for customizing performance: the encoding object, transcoding iterators, and customization points.
- Steve provided an example in which fast transcoding is trivial: transcoding ASCII to ISO-8859-1.
- Mark observed that programmers want fast functions and transcoding iterators, not encoding objects.
- Steve stated that, within iconv's implementation, all transcoding conversions go through Unicode code points for all encodings. This is presumably fast enough for most use cases. Converting from Shift-JIS to Big-5 doesn't require extreme performance.
- JeanHeyd stated that additional work is needed to enable that middle path with fast transcoding iterators.
- Tom agreed; we need the lowest level to fall back to enable transcoding iterators between all encodings, but can optimize specific cases.
- Zach stated that we really just need to list the specific transcoding iterators that are required.
- [P0267R9 - A Proposal to Add 2D Graphics Rendering and Display to C++](#):
 - Tom, unsurprisingly, stated that the interface should use `std::u8string` since it requires UTF-8 encoded text.
 - Michael agreed and expressed dislike for the assumption of UTF-8 in a `std::string` object.
 - Zach stated that the interfaces should be `std::string_view` and execution encoding.
 - Steve pondered whether all current graphical display systems are Unicode.
 - Tom stated that the X window system is locale based.
 - Zach suggested it would be least surprising to programmers to use execution encoding. That way they can just pass regular strings.
 - Peter stated that, On UNIX systems, UTF-8 tends to be the default, so things will work as is, but Windows would be problematic.
 - Zach observed that, without standard library support, converting text from execution encoding to UTF-8 is hard.
 - Peter suggested leaving it to the UI libraries to figure it out.
 - Zach responded that this is a UI library, so we need to figure it out.
 - Michael pondered whether we should add overloads for `char`, `wchar_t`, `char8_t`, `char16_t`, and `char32_t`.
 - Zach suggested that we only need `char` and `char8_t`.
 - Hubert observed that the standard library is designed around locales.
 - Tom asked Hubert to clarify, are you thinking these interfaces should take a locale object?
 - Hubert responded that, if you have strings that you don't know the encoding for, then yes.
 - JeanHeyd expressed a preference for just using `std::u8string` to avoid locale dependencies.
 - Mark agreed that, perhaps, just `char8_t` is enough.
 - Tom stated that, by the time 2D graphics is standardized, we should be able to get good conversion routines in the standard library or we will have failed miserably!
 - Hubert observed that the paper is missing bidirectional language support.
 - Tom noticed that the paper doesn't say what happens with ill-formed encoded input.
 - Mark suggested discussing font names; these should probably be bag-of-byte names. The paper defers to the HTML CSS specification.

- Zach noticed that the paper doesn't discuss normalization. It would be nice if it called it out specifically.
- Tom asked if normalization matters.
- Zach responded that it does in some cases.
- JF suggested that we should make it possible to defer to the CSS specification if we can't right now. We don't want to do what we previously did in forking the Unicode identifier specification from [UAX#13](#)
- Mark noticed that some of the interfaces pass and return `std::string` by value where they probably shouldn't.
- JF pondered about overlap with SG13 and avoiding conflicts in scheduling when meeting in Cologne.
- *[Editor's note: SG13 and SG16 are meeting on separate days.]*
- [P1750R0 - A Proposal to Add Process Management to the C++ Standard Library](#):
 - Elias described the overlap with [P1275](#) and stated he is aware of previous SG16 review and is working with Isabella Muerte.
 - Elias described the pipe interface.
 - Tom asked if any operating system supports wide pipes.
 - Elias stated he is unsure if Windows does. The interface is templated on char type.
 - Tom stated that Windows doesn't; `ReadFile` and `WriteFile` are used with pipes and they are byte oriented.
 - Hubert asked about the interaction with streams.
 - Elias responded that pipes can be wrapped in `iostreams`.
 - Tom summarized the feedback so far: wide pipes may not be needed and prior SG16 concerns regarding environment variables still stand.
 - Tom stated that command lines probably need to be considered to be in execution encoding.
 - Hubert stated that, for command lines, `exec` interfaces will likely be used and they use arrays, not strings. A formatting approach makes sense.
 - Elias stated that `process_launcher` takes a `std::filesystem::path`, not a string.
- Meeting in Cologne.
 - Tom communicated the tentative schedule for when SG16 would meet.
 - Zach stated he will miss Monday.

July 31st, 2019

Draft agenda:

- Cologne post-meeting discussion.
- Goals for WG14 in Ithaca (October 21st-25th).
- Goals for Belfast (November 4th-9th).

Attendees:

- Nathan Myers
- JeanHeyd Meneide
- Mark Zeren
- Steve Downey
- Tom Honermann
- Zach Laine

Meeting summary:

- Discuss drafting guidance explaining our consensus regarding providing char/wchar_t, char16_t, and char8_t overloads in Cologne.
 - Tom introduced the need to discuss guidance by presenting poll results taken for three papers:
 - [P1030R2](#): std::filesystem::path_view:
 - char and wchar_t oriented interfaces should be provided that behave according to the std::filesystem::path specification in terms of encoding.
S F F N A S A
3 2 0 4 2
 - char32_t oriented interfaces should be provided that behave according to the std::filesystem::path specification in terms of encoding.
S F F N A S A
2 2 4 2 2
 - [P0267R9](#): A Proposal to Add 2D Graphics Rendering and Display to C++
 - Provide overloads for char (execution encoding) and wchar_t.
S F F N A S A
0 0 4 3 3
 - Provide overloads for char16_t.
S F F N A S A
0 0 5 2 3
 - Provide overloads for char32_t.
S F F N A S A
0 0 3 3 4
 - [P1750R0](#): A Proposal to Add Process Management to the C++ Standard Library
 - Provide std::process char (execution encoding) and wchar_t interfaces.
S F F N A S A
7 2 0 0 0
 - Provide std::process char8_t interfaces.
S F F N A S A
3 0 5 0 0
 - Provide std::process char16_t interfaces.
S F F N A S A
0 0 8 1 0
 - Provide std::process char32_t interfaces.
S F F N A S A
0 0 3 5 0
 - Tom explained that, to an outside observer, our guidance looks inconsistent:
 - For polls about providing char and wchar_t based interfaces:
 - For P1030R2, we were evenly split with strong positions on both sides.
 - For P0267R9, we were fairly opposed to providing them.
 - For P1750R0, we were strongly in favor of providing them.
 - For polls about providing char16_t based interfaces:
 - For P1030R2, we didn't even ask the question (we know of UTF-16 based file systems).
 - For P0267R9, we were opposed to providing them.
 - For P1750R0, we barely could have cared less about the question.
 - For polls about providing char32_t based interfaces:
 - For P1030R2, we were evenly split with strong positions on both sides.
 - For P0267R9 and P1750R0, we were opposed (though more strongly so for P0267R9).
 - Zach addressed char32_t as the easy case first. The char32_t overloads exist for completeness, but no one actually uses them. They are inefficient. char32_t is more useful for interfaces that accept

non-contiguous data.

- Mark stated that `char32_t` is useful when examining Unicode scalar values or elements of a grapheme cluster.
- Zach replied that, If we have a grapheme cluster span like type some day, then we'll want a contiguous `char32_t` interface. We can always add `char32_t` overloads as needed later.
- Mark agreed that we can wait for use cases to materialize.
- Tom asked if we should consider deprecating any existing `char32_t` interfaces.
- Peter, despite not having been present for these polls and related discussion in Cologne, quickly recognized some patterns in the polls and offered some insightful rationale:
 - For P1750, we are replacing existing functionality, so need to support existing non-standard `char` and `wchar_t` based interfaces. `char8_t` is our intended future direction, so we want that interface. We don't want to emphasize `char16_t` and `char32_t` going forward.
 - For P0267, we are not replacing existing functionality, so we don't need `char`, `wchar_t`, `char16_t`, or `char32_t` based interfaces; we can restrict to `char8_t` for now.
 - For P1030, it seems like we don't know what we want.
- Mark added an additional rationale for P0267; fonts are Unicode based, so it makes sense to just start with Unicode input.
- Tom noted that, in the time since Cologne, Niall has decided to add `char` and `wchar_t` based interfaces to P1030.
- Zach expressed support for Peter's observations; `char` and `wchar_t` based interfaces are important for migration purposes.
- Mark agreed and noted that we don't want to construct road blocks for proposals for new interfaces.
- Peter acknowledged that we don't want to make migration difficult and then raised the point that Apple's HFS+ and APFS filesystems are problematic for `path_view` because their behavior is non-portable.
- Zach noted that similar problems exist for Windows with NTFS allowing UCS2 file names that are not valid UTF-16.
- Peter provided an additional example regarding FAT derived filesystems storing locale case translation tables and noted that this is problematic when files are written with one locale and read using a different one (probably on a different system).
- Tom returned to Peter's rationale in the context of P1030. What is being proposed is a more performant alternative for some uses of `std::filesystem::path`.
- Peter stated that the rationale for not providing `char` and `wchar_t` based interfaces is that the filesystem only offers bytes when names are enumerated. If we give those bytes back, the filesystem will accept them. We can get a displayable string, as from the `u8string()` member function of `std::filesystem::path`, but we can't necessarily pass that path back to the filesystem.
- Tom stated that that rationale contradicts guidance regarding not wanting to construct impediments to migration. The vast majority of file names use only the basic source character set. By not providing `char` interfaces, we're making very common use cases difficult.
- Zach observed that support for all valid file names requires use of `char` on Linux and `wchar_t` on Windows today. The goal of the `std::byte` oriented interface is to provide something portable.
- Tom objected to those interfaces providing a portable abstraction since:
 1. The underlying operating system interfaces used to implement those interfaces may themselves perform translations. For example, the normalization performed by HFS+ and APFS, and
 2. Some OS interfaces don't support arbitrary byte sequences as file names. For example, on Windows, a byte oriented interface would either use `CreateFileA` which would perform locale conversions, or `CreateFileW` which requires a sequence of 16-bit values (e.g., an odd number of bytes isn't supported).
- *[Editor's note: at this point, Tom became completely engrossed in the conversation and utterly and completely failed to record individual commentary. The following reflects his recollection of the discussion.]*
 - *Zach lol'd at the contortions that Tom's face apparently exhibited as Tom struggled to comprehend why anyone thought the `std::byte` based interface was a good idea.*

- *Tom was awakened to the possibility that the `std::byte` interface wasn't necessarily conceived of as a means to specify an actual sequence of bytes to be stored directly in the filesystem, but rather as a pointer to a sequence of bytes that represent an opaque structure that was (probably) provided by the OS in the first place.*

]

- Zach stated that `path_view` is intended for performance and doesn't support mutation.
- JeanHeyd asserted that the `std::byte` oriented interface is intended to allow passing back to the OS a path name that was originally provided by the OS.
- Zach agreed and added that the byte oriented interface is more like a handle to a file name, specifically a reference to something matching the representation stored in `std::filesystem::path`.
- JeanHeyd added that the byte oriented interface exists for performance, but the `char` and `wchar_t` interfaces should be provided for simple portable uses.
- Zach expressed a preference for making use of the `path_view` `char` based interface ill-formed on Windows and use of the `wchar_t` interface ill-formed everywhere else, but added he was now convinced that the `char` and `wchar_t` based interfaces should be provided.
- Mark observed that providing those means we need to worry about life-time management and when conversions occur.
- JeanHeyd responded that working implementations of `path_view` have already shipped and have demonstrated reduced overhead due to avoidance of allocation.
- Tom expressed a preference for introducing a `raw_path` type to represent a canonical path rather than using `std::byte`.
- JeanHeyd suggested using `std::filesystem::path::value_type` but noted that casts would still be needed.
- Zach pondered the idea of a `raw_path` type that is only constructible from `wchar_t` on non-Windows systems and only constructed from `char` elsewhere.
- Tom confirmed the date for our next telecon; August 21st with the intent being to discuss [P1108R2](#) - `web_view`.

August 21st, 2019

Draft agenda:

- Discuss P1108, "web_view". Our focus will be, unsurprisingly, character encodings and the use of iostreams with (presumably) UTF-8 data.
- Goals for WG14 in Ithaca (October 21st-25th).
- Goals for Belfast (November 4th-9th).
- Discuss a few follow up items from P1689, "Format for describing dependencies of source files", following discussion in SG15.
 - Bikeshed "data". What do we call the code unit equivalent in path names?
 - Are we ok stating that JSON readers/writers are not allowed to apply Unicode normalization?
 - Are we ok with allowing a BOM (JSON doesn't permit one)?
- Is "execution character set" the right term for the run-time locale dependent encoding used by the character classification and conversion functions?

Attendees:

- Corentin Jabot
- Hal Finkel
- Hubert Tong
- JeanHeyd Meneide

- Steve Downey
- Tom Honermann
- Zach Laine

Meeting summary:

- Discussion of a draft of P1108R3 - web_view:
 - <https://wg21.link/p1108r3>.
 - Hal introduces.
 - A prototype is available using wxWidgets:
 - https://github.com/hfinkel/web_view
 - There are a variety of ways we can provide graphical interaction within the standard.
 - This approach comes out of discussions with folks at Apple and Nvidia.
 - This approach outsources functionality to well used outside standards.
 - The basic idea is that system services already exist with different APIs that can be wrapped in a standard interface.
 - For security reasons, interactions should run out-of-process and the interface must therefore not be too fine grained.
 - There is a common subset of functionality among the various system services that provides a push/pull interface.
 - Constructing a web_view presents a window in which web content can be displayed and (Javascript) scripts can be run.
 - URI scheme extensions are supported by registering a (single) callback handler (per scheme).
 - Close handlers are supported by registering a (single) callback handler.
 - Interfaces are provided to request window close and to wait for window close.
 - An example of a dynamic page is available in the paper.
 - Hal provided a (successful!) live demonstration of the example from the paper.
 - Hal then provided an additional (successful!) live demo of an additional example.
 - Zach asked how C++ code can be invoked to update the displayed page.
 - Hal responded that interaction is enabled by registering a URI scheme handler callback via the `set_uri_scheme_handler` interface.
 - Tom asked if the interface is effectively append only.
 - Hal responded that it is based on a push model, so yes, requests update state. The design supports both push (via `run_script`) and pull (via callbacks registered with `set_uri_scheme_handler`).
 - Zach stated that users will want the ability to route schemes to direct requests.
 - Tom suggested that routing can be implemented via the callback registered with `set_uri_scheme_handler`.
 - Corentin suggested using Web Sockets as well.
 - Hal responded that there are many examples where utility libraries would come in helpful. For example, we probably don't want to do URI encoding and decoding, nor build interfaces using `std::format`. We probably want JSON support libraries. Such utility libraries should be proposed separately though.
 - Tom asked to clarify if `run_script` is for Javascript only and whether it would make sense for other languages to be supported.
 - Hal responded that it may be useful to specify the scripting language, like for Web Assembly.
 - Zach suggested that such support could always be wrapped in Javascript.
 - Zach acknowledged the elephant in the room by asking about the use of `std::string` in the interface.
 - Corentin stated that we should give the same advice as for 2D graphics; use Unicode everywhere and, specifically, UTF-8. Supporting both UTF-8 and UTF-16 would complicate the interface.
 - Zach noted that the W3C recommends UTF-8 only.
 - Zach observed that for support of [RFC 3986](https://tools.ietf.org/html/rfc3986), encoding of URIs could be handled within the library thereby allowing all URIs to be provided in UTF-8. The remaining interfaces could all take UTF-8

only as well, except, perhaps, for the window title.

- Tom stated that, for the title, even if UTF-16 is eventually required, conversion from UTF-8 is loss-less.
- Corentin suggested that URI escaping is complicated and that an interface for it should not be part of this proposal.
- Tom asked if existing web view providers provide URI encoding services or if the implementation would be obligated to provide it.
- Hal responded that some web view implementors just reject invalid URIs and that some others may not validate much for file handling. It isn't clear how existing web view providers interpret input; they probably just assume UTF-8.
- Hal asked that, if UTF-8 were required, would it be sufficient to indicate that by just using `std::u8string` in the interface.
- Zach responded yes, though `std::u8string` doesn't enforce well-formed UTF-8, so it may still be necessary to explicitly specify a requirement for well-formed UTF-8 data.
- Corentin asked if use of `char8_t` based types doesn't already ensure that.
- Hubert responded no, we can't enforce well-formedness since programmers can always create `char8_t` arrays with non-UTF-8 data.
- Zach suggested that we add blanket wording somewhere in the standard library specification stating that, for interfaces that use `std::u8string` in library functions, that behavior is undefined if data is not well-formed UTF-8.
- Hubert stated that approach makes sense.
- Hal, changing topics, asked for feedback regarding use of `std::ostream` in the URI scheme callbacks.
- Zach asked if we have `char8_t` based streams yet.
- Tom responded no.
- Zach stated that we would want that to help ensure the data is UTF-8.
- Hubert suggested that `codecvt` facets could be used to perform conversions.
- Zach acknowledged and added that, if the programmer imbues a locale, it is up to them to make sure it makes sense.
- Corentin asked if Hal had considered use of strings instead of streams?
- Hal responded that a string based approach might make sense. The benefit of the stream approach is that it allows partial writes and some of the lower level interfaces support that.
- Tom, clarifying, stated that, within a callback handler, data written to the stream may start being processed by the web view before the handler returns.
- Corentin suggested that we're going to have to provide `char8_t` based streams in C++23 anyway.
- Tom agreed.
- Hubert returned discussion to the earlier comments on blanket UTF-8 wording for `std::u8string`. The place to add such wording is in [res.on.arguments](#); "each of the following applies to all functions ... unless explicitly stated otherwise".
- Zach volunteered to draft that blanket wording.
- Hal stated that we kind of broke UTF-8 hello world in C++20, but `iostreams` are weird for non-text data anyway.
- Tom replied that it was already broken, but we certainly didn't make it any easier.
- Hubert noted that localizations on `iostreams` currently require characters not in ASCII. For example, monetary symbols like the Euro sign (€).
- Hal noted that the URI scheme handler takes a constrained parameter, so overloads could be provided to handle strings and streams.
- Hal stated that the next revision of the paper will include discussion about the URI scheme handler composing a string and returning it vs support for partial writes via `iostreams` or some other concept.
- Tom suggested that there may be something in the Networking TS worth looking at.
- Hubert suggested that something lower level in `iostreams`, like `std::streambuf`, might be worth looking at too.
- Hal observed that `std::streambuf` has an associated locale.
- Tom acknowledged; that is where `std::codecvt` facets do their work.

- Tom pondered whether we should ban `std::codecvt` facets on future `char8_t`, `char16_t`, and `char32_t` iostreams by making attempts to imbue such streams with such a facet an error.
- Tom mentioned that we've talked about string builders in the past and this is a clear example where such builders could be useful; though `std::format` might just be that tool these days.
- Zach observed that Beast and the like traffic in large ranges. Perhaps some of those types would be useful here.
- Corentin suggested that Web Sockets are a better solution.
- Tom asked if it might make sense for the URI scheme handler to just use Web Sockets.
- Hal responded with concerns about complexity; the underlying APIs aren't the same.
- Hal stated that we will need to figure out the string vs stream interface as we want to avoid having to do unnecessary copies. We don't want to motivate the interface based on not knowing how to print UTF-8 to streams. Responses are probably small, so strings are usually ok. But encoded images might get pushed through these interfaces as well.
- Zach asked how many URL scheme handlers can be active at a time; if we were reviewing for SG13, I would want to know how much data can get pushed.
- Hal responded that the interface currently feels quick from a human perspective, but measurements of throughput haven't been done yet.
- Hal followed up with some details of the prototype; wxWidgets has an unfortunate feature where all of the URI callbacks are called on the UI thread. That isn't desired since a slow handler blocks the UI. All of the underlying implementations support running handlers on non-UI threads. The prototype needs to be changed to further explore that.
- Hubert noted that an implementation could presumably host this as a single process where the C++ code is the plugin, so we can't necessarily assume a thread model.
- Hal responded that, on most systems, the straight forward implementation method has the UI driven by a thread in the same process, but the web content renderer code runs in a separate process driven by RPCs. This will determine performance characteristics.
- Discussion of goals for WG14 in Ithaca (October 21st-25th):
 - JeanHeyd stated that he is planning to attend and to bring papers for:
 - [nodiscard]
 - Additional conversion functions for `char` and `wchar_t`.
 - Support for C.UTF-8 as the default C locale.
 - Steve stated that the only thing that knows the encoding of `wchar_t` is the standard library and asked if any encodings other than UTF-16 or UTF-32 are used in practice.
 - JeanHeyd responded yes, AIX for Chinese locales uses Big-5.
 - Tom added that z/OS uses a wide EBCDIC.
 - Corentin asked what the motivation is for SG16 to add more conversion functions to C.
 - JeanHeyd responded that it allows C++ implementors to use features provided by C.
 - Tom suggested that it might be worth asking implementors what they would want and whether they would actually use C interfaces.
 - JeanHeyd acknowledged and stated he would ask.
 - Zach stated that such interfaces might be nice to have for C, but C interfaces can't achieve the performance that Bob Steagall demonstrated with his UTF-8 work.
 - JeanHeyd noted that, since these interfaces are based on NTBSs, they will need to check for null characters or know the string length ahead of time.
 - Zach suggested that, for performance, it may be worth only looking ahead 16 bytes at a time.
 - Tom stated that he is hoping to attend Ithaca and to bring papers for:
 - `char8_t`.
 - Make `char16_t/char32_t` string literals be UTF-16/32.
 - Named character escapes.
- Discussion of goals for Belfast (November 4th-9th).
 - Steve stated he would like to put together an initial pass at cleaning up terminology for encoding and character sets.
 - Hubert stated that he would be happy with SG16 bringing such a paper, but timing is bad for CWG given where C++20 is at.

- Tom stated he would like to bring a paper to enable a portable method of specifying that source files are UTF-8 encoded.
- JeanHeyd stated he is working towards getting funding to work nearly full time on the [P1629](#) standard text encoding paper.
- Tom asked JeanHeyd what we can do to help prove the design works well in practice and suggested porting some project to it to demonstrate that:
 - the interface works and fits existing use cases.
 - that code is better.
 - that performance is retained or improved.
- JeanHeyd responded that there are opportunities for a few checkpoints along the way. For example, CppCon where a presentation is currently planned.
- Tom asked for candidate projects that would be good for exercising the interface.
- JeanHeyd responded that he had previously tried with a chat server and that a text editor would be a good choice.
- Tom confirmed that the next meeting will be on September 4th.

September 14th, 2019

Draft agenda:

- Discuss Corentin's draft D1854R0 - Conversion to execution encoding should not lead to loss of meaning
 - <https://cor3ntin.github.io/posts/encoding/D1854.pdf>
- Discuss a few follow up items from [P1689](#), "Format for describing dependencies of source files" following discussion in SG15.
 - Bikeshed "data". What do we call the code unit equivalent in path names?
 - Are we ok stating that JSON readers/writers are not allowed to apply Unicode normalization?
 - Are we ok with allowing a BOM (JSON doesn't permit one)?
- Is "execution character set" the right term for the run-time locale dependent encoding used by the character classification and conversion functions?

Attendees:

- Corentin Jabot
- David Wendt
- JeanHeyd Meneide
- Nathan Myers
- Peter Bindels
- Steve Downey
- Tom Honermann
- Zach Laine

Meeting summary:

- The meeting started off with a round of introductions for the benefit of new attendees.
- Discuss Corentin's draft D1854R0 - Conversion to execution encoding should not lead to loss of meaning
 - <https://cor3ntin.github.io/posts/encoding/D1854.pdf>
 - Corentin introduced the paper:
 - The basic idea is to avoid the meaning of the program silently changing in unintended ways due to lack of representation in the execution character set for a character in a character or string literal.
 - Zach asked if he hadn't previously signed up to write this paper.

- Corentin explained that Zach signed up to write a paper about `u8string`.
- Tom then proceeded to explain the wrong paper but succeeded at only further confusing himself.
- Zach clarified that the paper he did sign up to write was to permit `u"xxx"` string literals only when the execution encoding is a Unicode encoding.
- Tom returned discussion to the paper at hand and noted that the paper only adds restrictions on ordinary and wide literals because restrictions are already in place for `u8`, `u`, and `U` literals.
- Corentin demonstrated via [godbolt.org](https://godbolt.org/z/kDwR1l) that `gcc` rejects non-representable characters and that `MSVC` substitutes a `?`.
 - <https://godbolt.org/z/kDwR1l>
 - *[Editor's note: demonstration of MSVC's substitution of a ? character requires adding the /source-charset:utf-8 option to the MSVC command line in the above link. Without that option, the UTF-8 encoded source is interpreted by the MSVC compiler as Windows-1252.]*
- Corentin summarized that the goal is to standardize `gcc`'s behavior.
- Corentin stated that he was unsure if Microsoft would be willing to implement this outside of `/permissive-` mode since this might break existing code even though such code is already fragile and subject to breakage just by being compiled on a different system (with a different default execution character set).
- Tom noted that by making this standard, if an implementor remains non-conforming, then users can complain if they want to.
- Tom asked if there are any possible advantages to status quo.
- Zach replied no, this just hurts portability.
- Corentin observed that code can always be updated to use an escape sequence instead of an unrepresentable character.
- Peter expressed concern about wide encoding because, on Windows, it is (or used to be) `UCS-2`, so emoji can't be represented.
- Tom restated Peter's point; there may be cases where graceful degradation is ok. E.g., losing emojis.
- Peter reported testing `gcc` and found that, in wide encodings, characters outside the `BMP` were lost when printing to the console.

```
int main() {
    std::wstring s = L"\U0001f4a9";
    std::wcout << s;
}
```

- Tom suggested that this is due to a `libstdc++` `iostreams` issue; wide characters are simply truncated when `std::wcout` writes them to `stdout`.
- Corentin demonstrated that `gcc` rejects wide string literals with characters not representable in the wide execution character set as well.
- Tom requested a quick walk through of the wording.
- Tom suggested to update the paper to use stable names for the sections to be updated since numbers change.
- Peter noted that, in section 5.13.3.8, the red text is missing strike through.
- Corentin commented that, until writing this paper, he was not aware of multi-character literals.
- Peter responded regarding a recent use case for them for a table driven switch handling approach:

```
uint32_t tableId = (table->Signature[0] << 24) |
                  (table->Signature[1] << 16) |
                  (table->Signature[2] <<  8) |
                  (table->Signature[3] <<  0);

switch(tableId) {
    case 'APIC':
        ...
}
```

- Tom expressed some initial surprise to see the proposed wording changes for octal and hex escapes, but concluded that they make sense.

- *[Editor's note: it would be helpful to add examples to the paper of code that would become ill-formed.]*
- Discuss a few follow up items from [P1689, "Format for describing dependencies of source files"](#) following discussion in SG15.
 - Bikes shed "data". What do we call the code unit equivalent in path names?
 - Tom introduced the naming concern. [P1689R0](#) used the name "data" to refer to the sequence of individual elements of a path. [P1689R1](#) changed the name to "code-units" following feedback in Cologne. Do we want to suggest a different name given our stance on file names not having an associated encoding and, arguably therefore, no "code units"?
 - Corentin argued to not invest time in this discussion unless/until SG15 progresses the paper further.
 - Corentin also observed that user's won't see this name, so it doesn't really matter.
 - Are we ok stating that JSON readers/writers are not allowed to apply Unicode normalization?
 - Tom explained that this is no longer a concern. in [P1689R1](#), code units are always explicitly specified.
 - Are we ok with allowing a BOM (JSON doesn't permit one)?
 - Corentin argued that we should follow the JSON specification.
 - Tom explained his understanding that allowing one doesn't violate RFC 8259 since the BOM limitations there only apply to network-transmitted text, and ECMA 404 doesn't specify encoding at all; there is no mention of "BOM", "byte order", or "UTF-8" in that specification.
 - <https://tools.ietf.org/html/rfc8259#section-8.1>
 - <https://www.ecma-international.org/publications/standards/Ecma-404.htm>
 - Zach asked what motivation exists for allowing a BOM.
 - Tom replied that it would be useful for non-ASCII based platforms like z/OS.
 - Peter added that it is useful for Windows as well since text files are likely to be interpreted as Windows-1252.
 - Corentin noted that Unicode recommends against use of a BOM.
 - Corentin stated that, if the specifications don't require UTF-8 encoded JSON, then we should specify that.
- Is "execution character set" the right term for the run-time locale dependent encoding used by the character classification and conversion functions?
 - Zach suggested asking core about this since it seems like we've just been using the wrong terms.
 - Steve noted that the existing wording is all old language pertaining to character sets, not necessarily encoding.
 - Tom stated that there was an email thread about this on the core and SG16 mailing lists and that the conclusion was that Steve and Tom should write a paper. Steve has since done some work, but Tom hasn't.
 - Zach stated that we need someone to go through the existing wording and refine our understanding of it.
 - Tom agreed, and added that that is the paper to be written. We use terms like "execution encoding" now that aren't defined in the standard.
 - Steve stated he would love to expose encoding details somehow.
 - Corentin asked if we want to change the names as they've been around a long time.
 - Steve stated he thinks it is worth tightening the specification without changing the intent. Other than that we should state that the wide character encoding can be a variable length encoding.
 - Zach commented that clarifying terms in the standard is a good use of our time.
 - Corentin stated we should have different names for compile-time and run-time encodings and that wording should state requirements regarding their compatibility.
 - Steve asserted that some archaeology is necessary here as much of this wording was created when locales were being developed around the expectation that code worked with the "C" locale.
 - Peter observed that variable length encodings go back to at least GB2313 from the 1980s.
 - Steve noted that shift encodings go back to then too.
- Zach mentioned that he has a repository where he is working on several small papers.
 - https://github.com/tzlaine/small_wg1_papers

- Peter requested feedback on his slides for CppCon.
- Tom stated that the next meeting will be September 25th.

September 25th, 2019

Draft agenda:

- Discuss LWG#3290 - Are std::format field widths code units, code points, or something else?
 - <https://cplusplus.github.io/LWG/issue3290>
 - Victor plans to have a draft paper for discussion.
- Discuss P1844R0: Enhancement of regex
 - <https://wg21.link/p1844>

Attendees:

- Corentin Jabot
- JeanHeyd Meneide
- Lyberta
- Mark Zeren
- Tom Honermann
- Victor Zverovich
- Zach Laine

Meeting summary:

- Discuss D1868R0 - 🌸 width: clarifying units of width and precision in std::format
 - <http://wiki.edg.com/pub/Wg21belfast/SG16/D1868R0.html>
 - Addresses <https://cplusplus.github.io/LWG/issue3290>
 - Victor introduces:
 - Any solution to this problem must deal with conflicting constraints. The programmer's intention is to align text output assuming a monospace font and some understanding of how the text will be rendered (e.g., how many terminal columns will be consumed by each "character"). Implementors desire a clear and precise specification; preferably one that does not have great complexity that may lead to reliability issues or bug reports.
 - Field precision is more consequential than field width because it truncates text potentially resulting in ill-formed output if truncation doesn't occur at a suitable boundary.
 - Experimentation with an approach that estimates field width based on Unicode's extended grapheme clusters and script blocks produced good results; better results than estimation based on code point counts.
 - Experimentation on macOS, Linux, and Windows revealed that Windows currently has the most significant limitations with regard to support for Unicode characters currently not represented in Microsoft's supported ANSI code pages. Experiments have not been performed using the new Windows terminal which may be expected to produce better results.
 - Testing of Unicode family emoji demonstrated the most variability of results since family emoji may be rendered as a single glyph or as a series of glyphs each representing a family member.
 - Field width is an estimate. Unless apriori knowledge of how the text will actually be rendered is available, the width of any given text can only be approximated.
 - The experimental implementation uses [Boost.text](#) to identify extended grapheme cluster boundaries and computes width based on Unicode block ranges culled from an implementation of wcswidth.

- Corentin mentioned that the issue with family emoji extends to other sequences of combining emoji. For example, ninja cat (U+1F431 {CAT FACE}, U+200D {ZERO WIDTH JOINER}, U+1F464 {BUST IN SILHOUETTE}) is rendered with a single glyph on Windows, but currently with two glyphs on Linux. Width fundamentally depends on rendering.
- Corentin added that, for non-Unicode encodings, width estimation must look at code units and do things differently for double-byte characters vs single-byte characters.
- Victor stated that he is content with handling of non-Unicode encodings being implementation defined.
- Zach agreed and asserted that we want a 90% solution. Support of non-Unicode encodings would require information that we can't currently specify in the `std::format` interface assuming `std::format` remains locale independent; it is ok for implementations to assume an encoding.
- Tom thanked Victor for doing this research and stated he found it sufficiently compelling to take the code unit solution he previously advocated for resolving [LWG 3290](#) off the table. In particular, the demonstration of prior art in the form of POSIX [wcswidth](#) lent confidence to this approach.
- Tom asked if width calculation for `wchar_t` could be delegated to `wcswidth`.
- Victor replied that `wcswidth` is locale dependent and that goes against the `std::format` design.
- Tom asked if width calculation for `char` and `wchar_t` couldn't be implementation defined such that an implementation could query locale only when width or precision is explicitly specified and the arguments are characters or strings. Width or precision specifiers would effectively constitute an opt-in for locale dependence.
- Zach objected on the basis that dependence on locale could cause output to differ on one platform vs another for the same character or string data.
- Victor clarified that, if encoding doesn't match, the worst case result is mis-alignment.
- Corentin stated that, as currently specified, `std::format` formats bytes since it doesn't know the precise encoding of inputs. Correct text manipulation requires knowing the encoding.
- Corentin expressed agreement that display width is what programmers expect. Perhaps in C++23, the ability to pass an encoding argument could be added.
- Tom mentioned that `std::format` can take a `std::locale` argument from which the encoding could be queried thus making it possible for programmers to opt-in to locale awareness simply by passing a locale object.
- Zach again objected based on the desire to have portable output.
- Corentin expressed a strong preference for a good solution in C++20 and asked if we could specify that width and precision units are display width and, for characters outside the basic source character set, behavior is implementation defined.
- Victor stated that is a minimum viable solution. The paper proposes that encoding is an implementation defined fixed encoding, not a run-time selected one.
- Corentin confirmed satisfaction with a minimal solution for C++20 that we can iterate on for C++23 and that retains some flexibility.
- Zach observed that, if we make it implementation defined today, then we'll be stuck with implementation choices. If the standard doesn't specify behavior, then implementors will choose one and we'll get stuck either way. This is similar to breaking ABI; it can be an over-my-dead-body issue.
- Corentin again expressed a desire for some way to preserve the ability to make changes later.
- Zach stated that it is important to remember what Victor said previously; width is an estimate.
- Mark observed that what we're discussing is mostly an edge case since most fields are aligned for numeric output.
- Tom countered that alignment is useful for things like names.
- Tom asked if `std::format` is `constexpr`.
- Victor replied that parsing of the format string is `constexpr`, but actual formatting is not.
- Corentin stated it would be useful to have `constexpr` formatting at some point, but querying locale would prevent that.
- Tom disagreed and stated that an implementation could use an internal locale if formatting at compile-time.
- Tom summarized his perceptions of our positions so far:

- We appear to have agreement for display widths in some form.
 - We have disagreements over adding a locale dependency as part of encoding assumption.
- Corentin asked Zach if he thought a best attempt at display width is sufficient.
- Zach replied that he wants the algorithm in the paper so that the same behavior is exhibited on all platforms and is unconcerned about rendering dependent cases like for family emoji.
- Victor reiterated that width calculation is best effort and that he is ok with consistent results only being ensured for the basic source character set. This assurance only requires a fixed system dependent encoding.
- JeanHeyd asked for clarification that we would only be guaranteeing alignment for the basic source character set in C++20 while leaving further specification until C++23.
- Victor replied, yes, basically.
- JeanHeyd asked if that implied an implementation defined fixed encoding.
- Victor responded, not implementation defined, but rather platform dependent so that all implementations targeting a given platform would exhibit the same behavior.
- Tom observed that, if the system defined fixed encoding differs by platform, then we won't get consistent results.
- Zach disagreed based on a premise that, for the purposes of width computation, consistent results are achieved by interpreting the input as Unicode.
- Corentin stated that he thinks we need to defer to (wide) execution encoding when computing width.
- Tom agreed stating that we should make width calculation as right as we can make it.
- JeanHeyd reformulated the trade off. The most right answer depends on locale. The always consistent result generates garbage consistently but avoids the locale dependency.
- Victor stated that rendering can always change; we just need to decide if we are ok depending on something at run-time.
- Zach re-iterated that, with the current specification, width calculation only works for single byte characters that render as a single glyph and we don't have a way to customize the width formatting unless we defer to something at run-time, but doing so conflicts with design goals of `std::format`.
- Corentin observed that the same issue exists with `printf` as it will fail if the execution encoding doesn't match the run-time locale encoding; C and C++ fundamentally depend on encoding compatibility.
- Victor reminded everyone that the paper does support use of the locale encoding via an opt-in specifier.
- Steve reminded everyone that there is no system call to get the actual display width, so we're always guessing anyway.
- JeanHeyd stated that he thought opt-in for locale dependent width was acceptable.
- Zach expressed a desire to get the right default for the long-term. If we make the default behavior locale sensitive, then we'll be stuck with that forever.
- Tom responded that, in the long term, encoding will hopefully become separated from locale thereby eliminating the wrong default concern.
- Corentin suggested that, for C++20, we could require the `'l'` in the specifier and not have a non-locale option until we figure this out.
- Steve observed that the locale dependency creates a buffer overflow situation in the case where the locale changes in between width calculation and actual formatting to a buffer.
- Corentin stated a preference to just require `'l'` in the width specification for C++20 to give us time to address this properly.
- Tom suggested adding a reference to [LWG 3290](#) in the paper.
- Tom announced that the next meeting will be on October 9th.